

TREE 2

PROBLEM 4

QUESTION:

DS	Session	Tree 2	Question Information	Level 1	Challenge 74
Problem Description You are given a weighted graph with N vertices and M edges. Find the total weight of its maximum spanning tree. Constraints • $1 \leq T \leq 20$ • $1 \leq N \leq 5000$ • $1 \leq M \leq 100\,000$ • $1 \leq c \leq 10\,000$ Input The first line contains one integer T denoting the number of test cases. Each test case starts with a line containing 2 space-separated integers: N and M. Each of the following M lines contain description of one edge: three different space-separated integers: a, b and c. a and b are different and from 1 to N each and denote numbers of vertices that are connected by this edge. c denotes weight of this edge. Output For each test case output one integer - the total weight of its maximum spanning tree.					

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 1 3 3 1 2 2 2 3 3 1 3 4 EXPECTED OUTPUT 7	INPUT (STDIN) 1 3 3 1 2 3 2 3 4 1 3 5 EXPECTED OUTPUT 9

Mandatory Test Cases

Test Case 1
KEYWORD int printheap(int N)

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 2	TOKEN COUNT 715	NLOC 110

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Connection {
    int src, dest, cost;
} Connection;

int sort_by_cost_desc(const void* a, const void* b) {
    return ((const Connection*)b)->cost - ((const Connection*)a)->cost;
}

int leader_node[5005];

int get_leader(int i) {
    if (leader_node[i] == i) return i;
    return leader_node[i] = get_leader(leader_node[i]);
}

int main() {
    int test_runs;
    if (scanf("%d", &test_runs) != 1) return 0;

    while (test_runs--) {
        int vertices, edges_count;
        scanf("%d %d", &vertices, &edges_count);

        Connection connections[edges_count];
        for (int i = 0; i < edges_count; i++) {
            scanf("%d %d %d", &connections[i].src, &connections[i].dest, &connections[i].cost);
        }

        qsort(connections, edges_count, sizeof(Connection), sort_by_cost_desc);

        for (int i = 1; i <= vertices; i++) leader_node[i] = i;

        long long total_max_weight = 0;
        int connected_edges = 0;
        for (int i = 0; i < edges_count; i++) {
            int root_u = get_leader(connections[i].src);
            int root_v = get_leader(connections[i].dest);
            if (root_u != root_v) {
                leader_node[root_u] = root_v;
                total_max_weight += connections[i].cost;
                connected_edges++;
                if (connected_edges == vertices - 1) break;
            }
        }
        printf("%lld\n", total_max_weight);
    }
    return 0;
}
```

OUTPUT:

```
1
3 3
1 2 2
2 3 3
1 3 4
7
```

```
1
3 3
1 2 3
2 3 4
1 3 5
9
```